

École Supérieure d'Informatique

First Year Preparatory Cycle
Algorithmics and Dynamic Data Structures

Set Theory on Linguistic Structures

Using Dynamic Data Structures in C

Submitted by

ABDELMOUMENE Hamza Ayoub
TALBI Baghdad

Supervisor: Dr. Kermi

Academic Year 2025–2026

| Deadline: 10 May 2026

Contents

1	Introduction	2
2	Objectives	2
3	Data Structures and Design Rationale	2
3.1	Three-Level Hierarchy	2
3.2	Binary Search Tree (Word Level)	3
3.2.1	Balancing Without AVL	3
3.3	Linked Lists (Sentence and Paragraph Levels)	4
3.3.1	Index Array Optimisation	4
4	Abstract Machine Conventions	4
4.1	Linked List Macros	4
4.2	BST Macros	5
5	Implementation	5
5.1	File Parser	5
5.2	Set Operations	5
5.3	Memory Management	6
6	User Interface	6
6.1	Architecture	6
6.2	Key Features	6
7	Testing and Validation	7
7.1	Test Files	7
7.2	Edge Cases Handled	7
8	Conclusion	7

1 Introduction

Set theory constitutes one of the most fundamental branches of mathematics, providing a rigorous framework for reasoning about collections of distinct objects. Formally, a **set** is a well-defined, unordered collection of elements, where membership is the primary relation of interest.

Given two sets A and B defined over a common universe U , the three principal operations are:

$$A \cup B = \{x \mid x \in A \vee x \in B\} \quad A \cap B = \{x \mid x \in A \wedge x \in B\} \quad A \setminus B = \{x \mid x \in A \wedge x \notin B\}$$

This project bridges classical set theory with computational linguistics by representing text files as *hierarchical set structures* implemented through dynamic data structures in C. Each file is decomposed into paragraphs, each paragraph into sentences, and each sentence into a set of unique normalised words. Standard set operations are then applied at each level of this hierarchy.

2 Objectives

The primary objective is to implement, in the C programming language, a complete pipeline that:

1. Reads and pre-processes one or more plain text files.
2. Decomposes each file into a three-level linguistic hierarchy (paragraphs $\rightarrow$ sentences $\rightarrow$ words).
3. Represents each level using an appropriate dynamic data structure.
4. Implements union, intersection, and difference at all three levels.
5. Presents results through an interactive terminal interface.

3 Data Structures and Design Rationale

3.1 Three-Level Hierarchy

The project represents the content of a file as a nested structure:

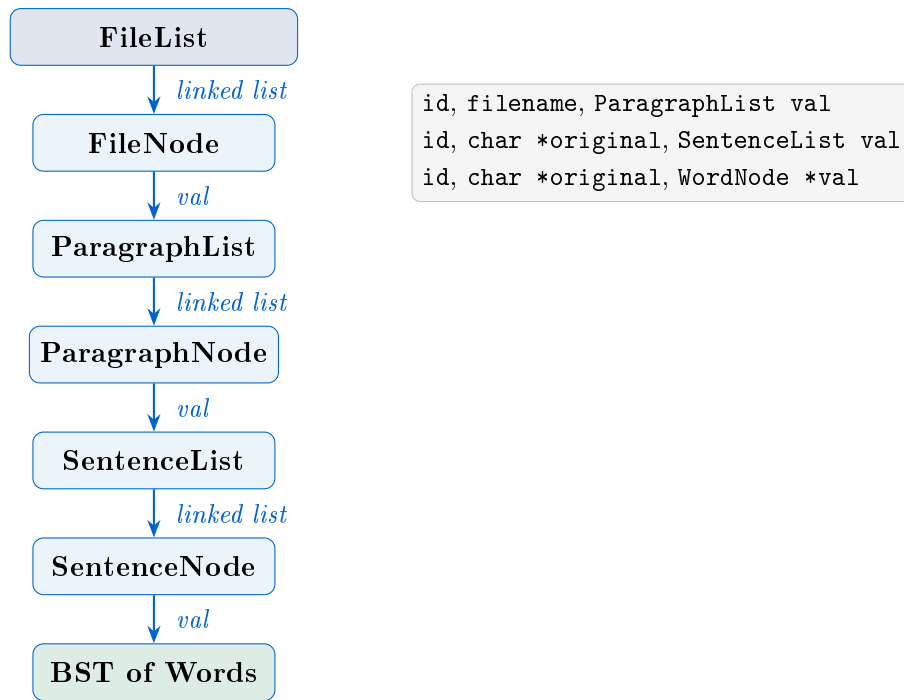


Figure 1: Three-level hierarchical data structure

3.2 Binary Search Tree (Word Level)

A BST is chosen for the word set because:

- Duplicate rejection is implicit — inserting an existing word is a no-op.
- In-order traversal produces alphabetically sorted output with no extra work.
- Search and insertion are $O(\log n)$ when the tree is balanced.

3.2.1 Balancing Without AVL

Rather than implementing AVL rotations, the project uses a **two-pass balanced build**:

1. Collect all words into a dynamic `char**` array during parsing.
2. Sort the array alphabetically with `qsort`.
3. Insert in *median-first* order recursively.

Inserting the median of a sorted array first guarantees height $\lceil \log_2 n \rceil$ — identical to a perfectly balanced BST — with no rotation code.

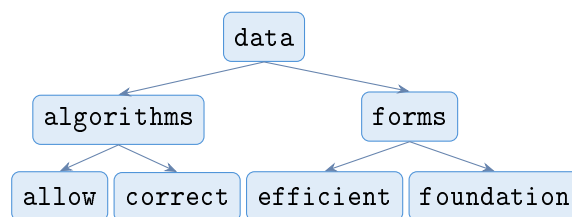


Figure 2: Example BST built by median-insert on a sorted word array

3.3 Linked Lists (Sentence and Paragraph Levels)

A singly-linked list with a maintained **tail** pointer provides $O(1)$ append, which is the primary operation during parsing. Because the number of sentences and paragraphs is unknown before reading the file, a static array would require over-allocation or costly reallocation.

3.3.1 Index Array Optimisation

After parsing completes, a **BuildIndex** pass walks the list once and stores each node pointer in a parallel **Node** index** array. This converts all subsequent random accesses (menu selection, set operations) from $O(n)$ linear scan to $O(1)$.

Table 1: Data structure choice rationale

Level	Structure	Insert	Search	Reason
Word	Balanced BST	$O(\log n)$	$O(\log n)$	Set uniqueness, sorted output
Sentence	Linked list + index	$O(1)$	$O(1)^*$	Dynamic size, index after parse
Paragraph	Linked list + index	$O(1)$	$O(1)^*$	Dynamic size, index after parse

* After **BuildIndex**; $O(n)$ before.

4 Abstract Machine Conventions

The implementation strictly follows the "Machine Abstraite" macros required by the course. This ensures that data structure operations are expressed through logical primitives rather than raw pointer arithmetic.

4.1 Linked List Macros

Table 2: Linked list abstract machine

Macro	Expansion	Role
Allocate (p)	(p) = CheckedMalloc (sizeof(*(p)), ...)	Allocate node
Free (p)	free (p)	Release node
Ass_val (p,v)	(p)->val = (v)	Set value field
Ass_adr (p,q)	(p)->addr = (q)	Set next pointer
Value (p)	(p)->val	Read value
Next (p)	(p)->addr	Read next

4.2 BST Macros

Table 3: Binary search tree abstract machine

Macro	Expansion	Role
<code>AllocateNode()</code>	<code>AllocateNodeImpl()</code>	Allocate tree node
<code>FreeNode(p)</code>	<code>free(p)</code>	Release tree node
<code>Ass_LC(p,q)</code>	<code>(p)->left = (q)</code>	Set left child
<code>Ass_RC(p,q)</code>	<code>(p)->right = (q)</code>	Set right child
<code>LC(p)</code>	<code>(p)->left</code>	Read left child
<code>RC(p)</code>	<code>(p)->right</code>	Read right child
<code>Ass_Node_Val(p,v)</code>	<code>strcpy((p)->val, (v))</code>	Store word string
<code>NodeValue(p)</code>	<code>(p)->val</code>	Read word string

5 Implementation

5.1 File Parser

The parser processes the input file character by character, maintaining a `ParserState` structure that accumulates the current word, sentence, and paragraph. The dispatch logic is summarised below:

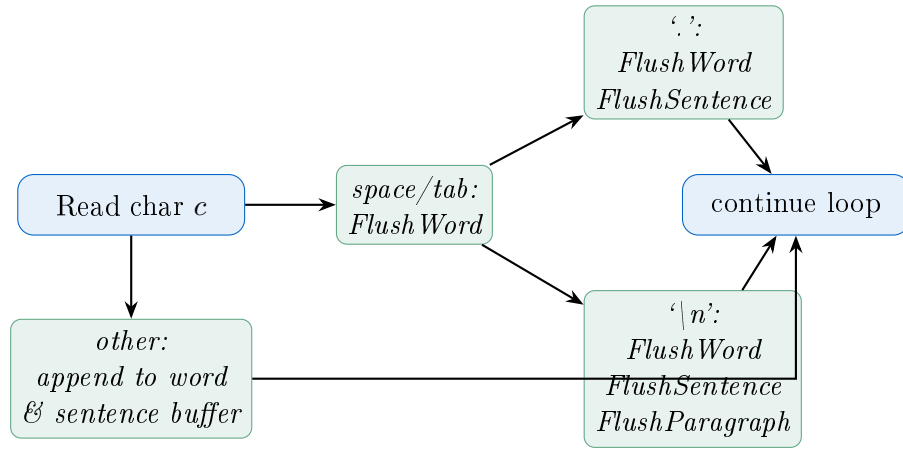


Figure 3: Parser character dispatch logic

The two-pass sentence build ensures every sentence BST is balanced: **(1)** collect words $\rightarrow$ **(2)** sort with `qsort` $\rightarrow$ **(3)** `MedianInsert`.

5.2 Set Operations

All nine operations follow a consistent pattern built on three comparison helpers:

- `BSTsEqual(A,B)`: collects both word arrays, compares element by element.
- `SentenceExistsIn(bst, list)`: walks the list, calls `BSTsEqual`.
- `ParagraphExistsIn(sentences, list)`: walks the list, calls `SentenceListsEqual`.

Table 4: Set operation implementations

Operation	Algorithm
WordUnion	Copy A into result; copy B — BST insert ignores duplicates
WordIntersection	Collect A's words; keep only those found by Search in B; MedianInsert
WordDifference	Collect A's words; keep only those <i>not</i> found in B; MedianInsert
SentenceUnion	Deep-copy A; append B sentences not already in result
SentenceIntersection	Append A sentences that exist in B
SentenceDifference	Append A sentences that do not exist in B
ParagraphUnion	Deep-copy A; append B paragraphs not already in result
ParagraphIntersection	Append A paragraphs that exist in B
ParagraphDifference	Append A paragraphs that do not exist in B

5.3 Memory Management

A centralised error-handling layer in `utils.c` ensures no allocation failure goes undetected:

- `CheckedMalloc` and `CheckedRealloc` abort on out-of-memory rather than returning `NULL`.
- All structures implement cascading `Free*` functions: `FreeFileList` $\rightarrow$ `FreeParagraphList` $\rightarrow$ `FreeSentenceList` $\rightarrow$ `FreeTree`.
- All `Free*` functions zero all pointer fields after freeing to prevent use-after-free.

6 User Interface

6.1 Architecture

The terminal interface is built around a centered 100-character box rendered with Unicode block characters. Two layout modes are used:

- **Dual-pane box** (`DrawBox`): left pane displays the ESI / ADDS / TP ASCII art; right pane holds 24 content slots.
- **Full-width box** (`DrawFullBox`): single 98-column pane for file preview and result screens.

6.2 Key Features

- **Single-keystroke navigation**: arrow keys, digit shortcuts, and ESC, all handled via raw terminal mode (`ReadChar`).

- **Bilingual interface:** pressing L at any menu toggles between English and French. All strings are stored in a compile-time `Str` table selected at runtime by a global `g_lang` flag.
- **Persistent file-loading screen:** after loading a file, the screen remains active for additional loads rather than returning to the main menu.
- **Paginated file preview:** paragraphs are displayed 12 per page with the first 10 words as preview text followed by ellipsis.
- **Animated loading bar:** a centered bordered progress panel with spinner and gradient fill runs during file parsing and set operations.

7 Testing and Validation

7.1 Test Files

Two test files are provided:

- `tests/test1.txt`: 36 paragraphs (4 repeated groups of 9 single-line paragraphs). Stress-tests pagination, paragraph preview truncation, and set operations on repeated content.
- `tests/test2.txt`: 3 paragraphs with 3 sentences each. Provides a second dataset for cross-file operations and subset checks.

7.2 Edge Cases Handled

- Empty file $\rightarrow$ empty `ParagraphList`, UI shows appropriate message.
- Paragraph ending without ‘.’ $\rightarrow$ flushed as one sentence at ‘\n’.
- File not ending with ‘\n’ $\rightarrow$ EOF flush sequence handles remaining state.
- All-punctuation word $\rightarrow$ normalised to empty string, rejected by `IsBlank`.
- Empty intersection $\rightarrow$ result displays $\emptyset$.

8 Conclusion

This project demonstrates a complete and coherent application of classical data structures to a linguistic processing problem. The Binary Search Tree provides natural set semantics for the word level; linked lists with parallel index arrays handle the sentence and paragraph levels dynamically while preserving $O(1)$ random access. The median-insert balancing strategy achieves optimal BST height without the implementation overhead of AVL rotations.

The interactive terminal interface adds bilingual support, animated feedback, and a structured navigation system, resulting in a polished deliverable that satisfies all requirements of the ADDS lab examination.